ValidaTISS – Revisão técnica sênior

Oct 26, 2026 · @Alex Soares

Resumo executivo

Veredito: não publicar o código original em produção. Qualquer pessoa na internet podia virar ADMIN, ler e alterar dados de qualquer cliente, gastar tokens de IA alheios e forjar um pagamento PIX. Entreguei uma versão corrigida (backend, frontend e testes) que fecha esses buracos; ela precisa de teste em staging com o banco real antes de ir ao ar.

Os 5 problemas que impedem o deploy:

1. **Sessão forjável.** O login era só um JSON em `sessionStorage`; digitar `{perfil:'ADMIN'}` no console abria o backoffice. Quase nenhum endpoint verificava quem chamava.
2. **`registro_admin` público.** Qualquer um criava um administrador via `/api/auth`. Havia também a senha provisória fixa `Mudar@123`.
3. **Webhook do Asaas sem autenticação.** Um POST forjado marcava campanhas e assinaturas como pagas; o fluxo ainda não tinha idempotência nem conferência de valor.
4. **IDOR e débito de tokens sem dono.** `/api/regras`, `/api/auditoria` e `/api/gestao_usuarios` aceitavam `clienteId` / `email` do corpo da requisição; o saldo era lido e debitado em passos separados (corrida).
5. **XSS e XXE.** Nomes, temas e textos de anúncio entravam em `innerHTML`; o parser de XML resolvia entidades externas.

O que foi entregue: 13 endpoints reescritos sobre um núcleo comum (`vt_core`), frontend sem HTML inline e compatível com CSP, `admin.html` refeito, 23 testes de API, um teste de ponta a ponta no Chromium e o README com a ordem de deploy. Dois arquivos que eu não recebi limitam a análise (ver "Limites da análise").

Urgente (corrigir antes de qualquer deploy)

#	Arquivo original	Risco	Correção entregue
1	`auth.js`, todos os `api/*.py`	Sessão em `sessionStorage`; endpoints sem autenticação/autorização	JWT em cookie `__Host-` HttpOnly/Secure/SameSite=Strict, `tokenVersion` conferido no banco, matriz de permissões por perfil (RBAC) em `vt_core/web.py`

#	Arquivo original	Risco	Correção entregue
2	`auth.py`	`registro_admin` aberto; senha fixa `Mudar@123`; erros revelam se o e-mail existe	Criação de interno só por ADMIN, senha provisória aleatória exibida uma vez e troca obrigatória; mensagem única de login; bloqueio de 15 min após 5 falhas
3	`webhook_asaas.py`	Pagamento forjado; reentrega duplica plano	Token no cabeçalho, chave primária `PagamentoProcessado` (idempotência) dentro de transação, conferência de valor e de vínculo cobrança→pedido
4	`gerar_pix.py`, `assinatura.py`	Valor e referência vinham do navegador; `assinatura` atualizava a tabela de funcionários	Valor sempre lido do banco/ambiente; `assinatura` só para staff e na tabela `cliente`, com auditoria
5	`regras.py`, `auditoria.py`	IDOR (qualquer `clienteId` / `email`); saldo lido e debitado separadamente	Identidade só pela sessão; débito atômico `update_many(... gte 1)` com estorno se a IA falhar
6	`validador.py`	XXE (lxml resolve entidades); versão desconhecida caia no XSD 4.03.00	Parser sem entidades/DTD/rede, rejeição de `<!DOCTYPE`, XSD em cache, versão declarada em `Padrao`
7	`admin.js`, `ui.js`, `cliente.js`	XSS por `innerHTML` com dados de usuários e anúncios; `javascript:` em links	Construção via DOM (`el()`), `esc()`, `urlSegura()` só http/https, CSP `script-src 'self'`
8	`carga_tuss.py`, `dominio_tuss.py`	Escrita e exclusão de domínios sem login; tabela sem lista permitida; exclusão física	Permissões `tuss:*`, lista de tabelas, limite de 5.000 linhas por pacote, inativação reversível e auditada
9	`gestao_usuarios.py`	Sem autenticação; `atualizar_tokens` inexistente; reset para senha fixa; bloqueio sobrescrevia `perfil`	Reescrito com proteção do último admin e de autoedição, débito atômico e trilha em `LogAdmin`

Na revisão final achei e corrigi mais um furo meu: o *logout* só apagava o cookie, então um token copiado seguia válido por 8 h. Agora o logout incrementa `tokenVersion` (encerra também outros dispositivos) e há teste para isso.

Importante

Integridade e fluxo interface × API × banco

- A grade de clientes mostrava sempre FREEMIUM e 0 tokens: a API não devolvia `plano`, `statusPagamento` nem `tokensRestantes`. Corrigido em `gestao_usuarios`.

- `CampanhaAds.usuarioId` guarda, na prática, o `Cliente.id`; confirme no seu `schema.prisma` e, se der, crie a relação com chave estrangeira (sugerida em `prisma/schema.sugerido.prisma`).
- Os botões de publicidade do admin ("Orçar", "Aprovar exibição") eram simulação com dados fixos e `alert`. Agora gravam de verdade, com máquina de estados: PENDENTE → ORCADO → AGUARDANDO_PAGAMENTO → ATIVO, e o pedido encerrado não reabre.
- Cancelamento de conta ignorava a data efetiva; agora ela vale (a conta continua ativa até a data) e o cliente é desconectado ao ser bloqueado.
- Datas ingênuas (sem fuso) em campanhas e cargas TUSS viraram UTC com fuso.
- Faltam chaves estrangeiras, índices e unicidade na carga TUSS; veja `prisma/migracao_sugerida.sql` (testar em staging).

Desempenho e custo

- Cada requisição criava `Prisma()` + connect/disconnect. Agora há um cliente por processo e a `DATABASE_URL` deve apontar para o pooler. Isso elimina o `sleep` de 1,5 s entre pacotes da carga TUSS.
- Busca TUSS por `contains` no código (varredura completa) virou `starts_with` com limite de 50 e índice trigram opcional na descrição.
- `/api/campanhas` agora filtra datas no banco e responde com `Cache-Control: s-maxage=60`, poupando uma consulta por visitante a cada 8 s de rodízio.
- XSD compilado uma vez por instância (`lru_cache`), não a cada requisição.
- Limite de 4,5 MB de corpo da Vercel: XML grande falha no servidor. A saída de médio prazo é validar no navegador (XSD em WebAssembly) e enviar ao servidor só o que precisa.
- Chamadas de IA e Asaas ganharam *timeout* e erro genérico (antes um provedor lento segurava a função até o limite).

UX e acessibilidade

- **Feedback.** `alert()` e `prompt()` bloqueantes foram trocados por avisos não bloqueantes (`aria-live`) e por modais; botões mostram "Salvando…" e ficam desabilitados durante a chamada (fim do duplo clique que criava dois pedidos).
- **Erros claros.** Antes, falhas de rede não diziam nada (`catch(e){}`). Agora toda chamada informa o motivo ou pede para tentar de novo.
- **Senhas.** A política (10+ caracteres, 3 tipos, sem senhas comuns) é imposta no servidor e explicada na tela; a senha provisória do admin sai da tela depois de copiada.

- **Teclado e leitores de tela.** Esc fecha modais, o foco volta ao botão de origem, Enter envia login e buscas, campos têm `label` / `aria-label`, há "pular para o conteúdo" e foco visível. Os anúncios são `role=link` com teclado.
- **Contraste.** Texto branco sobre `#28a745` e `#f39c12` ficava abaixo de 4,5:1 (WCAG AA); os botões e selos foram escurecidos.
- **Movimento.** O rodízio e as animações respeitam `prefers-reduced-motion` e pausam com a aba oculta.
- **PIX.** O QR de demonstração usava a Google Chart API (descontinuada). Agora exibe o QR real do Asaas e o código copia-e-cola.
- **HTML.** O `index.html` tinha a barra de login fora do `<body>` e conteúdo depois de `</body>`; o `admin.html` tinha `<div>` fechada cedo e IDs duplicados (`edit-tuss-termo`, `edit-tuss-anvisa`). Corrigidos.
- **Mobile.** O `dashboard_cliente.html` desabilitava o zoom (`user-scalable=no`); removido. Modais ocupam 94% da largura em telas pequenas.
- **Pendência de produto.** "Comprar mais tokens" ainda não tem fluxo de compra no backend; o botão só leva ao início.

TISS, IA e LGPD

TISS

- **Hash do epílogo.** O original calculava MD5 do C14N de `prestadorParaOperadora` e o desativou. Implementei a regra do padrão como eu a conheço (MD5 dos textos fora do epílogo, em ISO-8859-1), no servidor (`vt_core/tiss.py`) e no navegador (`tiss_hash.js`, com teste de paridade). Tenho cerca de 85% de certeza da regra exata, por isso o servidor só **avisa** quando diverge. Valide com um arquivo aceito pela sua operadora antes de tornar o hash bloqueante.
- **Edição do XML.** Quem edita e salva um XML precisa recalcular o hash; sem isso a operadora recusa o lote. O `tiss_hash.js` já serve para isso, mas ligação ao `validador.js` depende do arquivo que não recebi.
- **Versão.** O XML declara a versão em `Padrao`; o servidor valida contra ela e avisa se difere da escolhida na tela (antes caía em silencio no XSD 4.03.00).
- **Encoding.** Declaração `ISO-8859-1` em texto já decodificado corrompia acentos; a declaração é removida antes do parse.
- **Erros.** A validação devolve linha, coluna e caminho (até 200 erros), mantendo 200/400 para não quebrar o front atual.

IA de auditoria

- Chave do Gemini passa no cabeçalho (na URL ela vazava em logs); *timeout* de 5 s + 40 s; erros do provedor não chegam ao navegador.
- Os itens vão como dado em JSON e o *prompt* manda ignorar instruções contidas neles (mitiga *prompt injection* vinda de campos do XML). A saída é forçada a JSON e normalizada antes de voltar à tela.
- Provedor só pode ser um dos habilitados por variável de ambiente e com chave configurada. Máximo de 60 itens por análise.
- Saída é apoio à decisão: a resposta traz o aviso de que não substitui o auditor.

LGPD (dados de saúde)

- Só código, descrição, quantidade e valor do item, mais a especialidade e o tipo de guia, vão ao provedor. Não enviamos número de guia (fica só no log interno), nome nem identificador do paciente. Confirme que o `validador.js` também não os inclui no `itens`.
- O cadastro agora exige aceite de Termos e Política de Privacidade, com data gravada (`aceiteTermosEm`). Os textos jurídicos precisam ser redigidos por você ou por seu jurídico.
- Provedores de IA em outro país são transferência internacional de dados; vale checar contrato e base legal com o DPO.
- `LogIA` guarda quem usou, qual guia e qual motor, sem o conteúdo; `LogAdmin` registra as ações de staff (sem senhas).
- Banners enviados por clientes são validados por *magic bytes* (PNG/JPEG), limitados a 300 KB e servidos com `CSP: sandbox`.

O que foi corrigido

Arquivo entregue	Substitui	Observação
`vt_core/*` (config, security, web, validation, db, audit, asaas, ia, tiss)	— (novo)	Núcleo comum: sessão, RBAC, CSRF, erros padronizados, validação
`api/auth.py`	`auth.py`	Login, registro, `me`, logout com revogação, troca de senha
`api/gestao_usuarios.py`	`gestao_usuarios.py`	IAM e carteira de clientes, com paginação

Arquivo entregue	Substitui	Observação
`api/assinatura.py`, `gerar_pix.py`, `webhook_asaas.py`	idem	Pagamentos com valor e vínculo vindos do servidor
`api/publicidade.py`, `campanhas.py`, `banner.py`	idem (+ `banner` novo)	Estados do anúncio e arte servida com segurança
`api/dominio_tuss.py`, `carga_tuss.py`	idem	Staff com permissão; carga em lote idempotente
`api/regras.py`, `auditoria.py`, `validador.py`	idem	Posse da regra pelo cliente; débito atômico; XXE fechado
`js/modulos/core.js`	— (novo, carregar primeiro)	Cliente de API, `esc()`, avisos, modais, despachante de eventos sem inline
`ui.js`, `auth.js`, `cliente.js`, `admin.js`	idem	Reescritos sobre o `core.js`; `admin.js` agora grava de verdade
`index.html`, `dashboard_cliente.html`, `admin.html`	idem	Estrutura corrigida; `admin.html` refeito
`css/style.css`	idem	Original preservado + bloco "VT-EXTRAS" (contraste, foco, avisos)
`prisma/schema.sugerido.prisma`, `migracao_sugerida.sql`	—	**Reconstruídos** a partir dos campos usados; comparar com o seu schema
`vercel.json`, `requirements.txt`, `scripts/criar_admin.py`	—	Cabeçalhos de segurança e CSP; dependências; primeiro admin

Testes que rodei: 23 de API com banco falso em memória (autenticação, CSRF, IDOR, RBAC, débito e estorno de tokens, webhook idempotente, XXE, revogação de logout), 1 roteiro de ponta a ponta no Chromium (31 verificações, incluindo XSS e violações de CSP)

e a paridade do hash servidor × navegador. Eles usam um `bcrypt` de mentira e não substituem um teste em staging com Postgres real.

Ordem segura de deploy

1. Backup do banco e comparação do schema sugerido com o seu; aplicar as colunas novas em staging.
2. Copiar os XSD TISS para `schemas/`.
3. Definir `SESSION_SECRET` (32+ caracteres), `DATABASE_URL` do pooler, `ASAAS_*`, `PRECO_PREMIUM_MENSAL` e as chaves de IA.
4. Criar o primeiro admin com `scripts/criar_admin.py`.
5. Deploy em *Preview* e conferir que `vt_core/` e `schemas/` foram empacotados.
6. Reapontar o webhook do Asaas (com o token) e pagar um PIX de R$ 1 em sandbox.
7. Promover a produção. Todos precisarão entrar de novo, o que é esperado.

Roadmap de evolução

Onda	Prazo sugerido	O que entra	Por quê
1. Colocar no ar com segurança	1 a 2 semanas	Staging com Postgres real; ajustar o schema; ligar o `validador.js` ao novo `/api/auditoria` e ao `tiss_hash.js`; XSD em `schemas/`; teste do PIX em sandbox; textos de Termos e Privacidade	Fecha as lacunas que eu não consegui testar e libera o deploy
2. Robustez operacional	3 a 6 semanas	*Rate limit* por IP no login e na IA (Vercel WAF ou Upstash); alertas de erro e de webhook divergente; tela de auditoria (`LogAdmin`); recuperação de senha por e-mail; MFA para ADMIN; paginar listas; consolidar os 3 blocos duplicados de CSS em um design system	Reduz o risco de abuso e o custo de suporte

Onda	Prazo sugerido	O que entra	Por quê
3. Produto	2 a 3 meses	Validação XSD no navegador (WebAssembly) para arquivos acima de 4 MB; compra de pacotes de tokens via PIX; relatório de glosas por operadora; testes de regressão com guias reais anonimizadas; *feature flag* para novos provedores de IA	Remove o limite de tamanho e abre novas receitas

Decisões que são suas: o tempo de bloqueio de login (hoje 15 min após 5 falhas, o que permite a um terceiro travar uma conta por alguns minutos), a validade da sessão (8 h) e se o hash do epílogo passa a bloquear ou continua só avisando.

Limites da análise

- **`validador.js` não foi enviado.** É o coração do front (regras, extração, edição do XML). Não revisei o motor de regras nem a edição. Ele provavelmente ainda chama `/api/auditoria` com o contrato antigo (e-mail no corpo) e precisa passar `{provedor, guia, itens, contexto}`, tratar 401 e 402 e recalcular o hash após editar.
- **`schema.prisma` não foi enviado.** O arquivo em `prisma/` é uma reconstrução. Nomes de campos e modelos (ex.: `usuarioId` guardando `Cliente.id`, `cnpj` × `cpfCnpj`, onde ficam `plano` e `statusPagamento`) podem diferir do seu.
- **`vercel.json` e os XSD TISS não vieram.** Não posso garantir o empacotamento de `vt_core/` e `schemas/` na Vercel; confirme em *Preview*. Sem XSD, `/api/validador` responde erro 500.
- **Testes com banco falso.** Os 23 testes de API e o roteiro do Chromium usam um Prisma em memória e um `bcrypt` simulado (o PyPI estava bloqueado). O comportamento real de transações, `update_many` e pooler só se confirma em staging.
- **Hash do epílogo** com cerca de 85% de certeza; ver a seção de TISS.
- **Revisão independente.** Tentei uma segunda leitura de segurança por outro agente, mas ela foi interrompida por limite de sessão; a revisão do núcleo de autenticação foi minha. Recomendo um *pentest* antes de abrir ao público.
- **Não assessoro juridicamente.** Termos de Uso, base legal LGPD e transferência internacional de dados precisam do seu jurídico/DPO.

- **Função "Ads" do admin em nome do cliente** foi removida: a API agora só cria pedidos para o próprio cliente logado. Se precisar, dá para reintroduzir como ação de staff com a permissão `ads:criar`.